

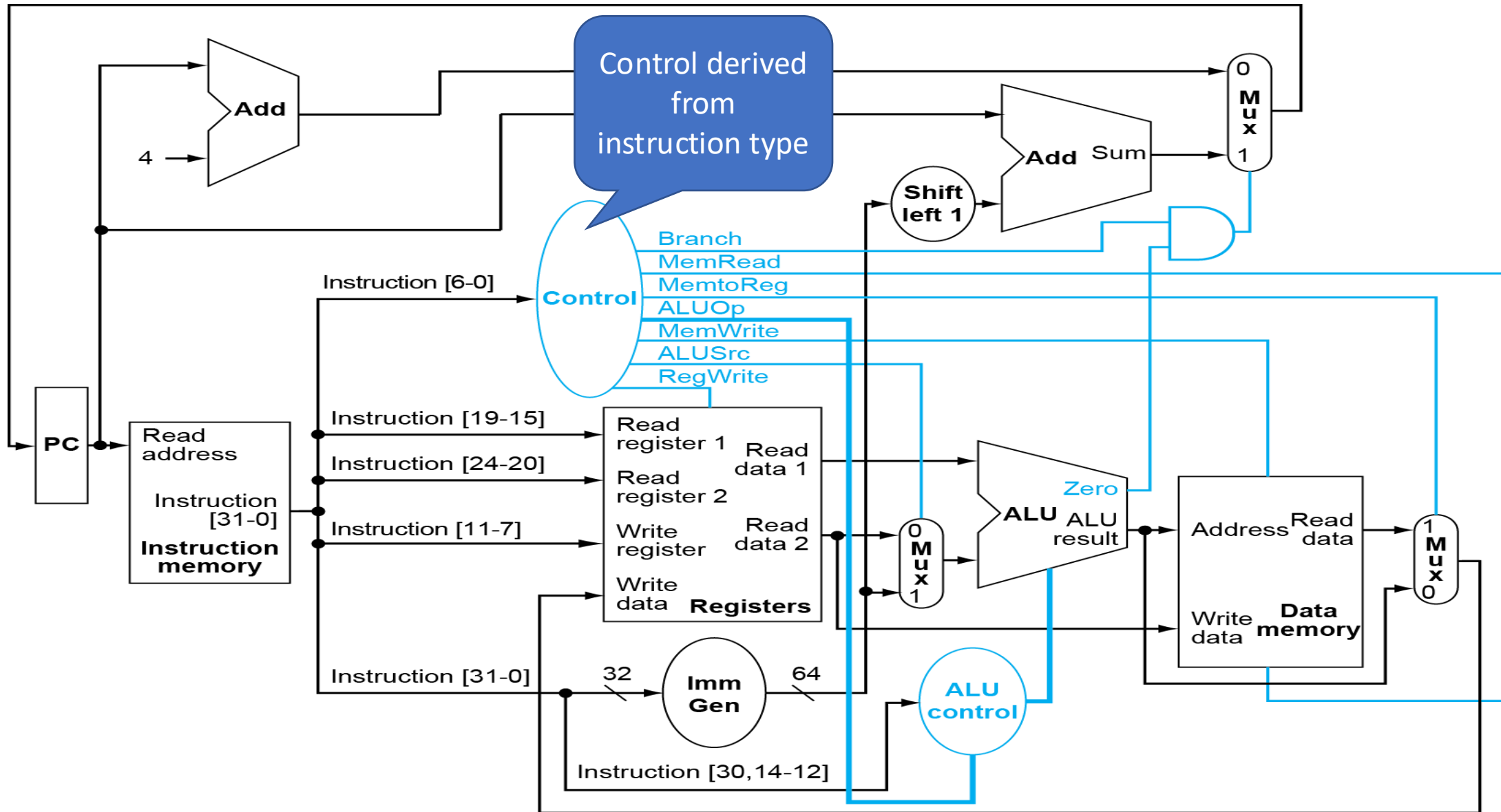
Basic/Pipelined processor Implementation

Jinyang Li

Today's lesson plan

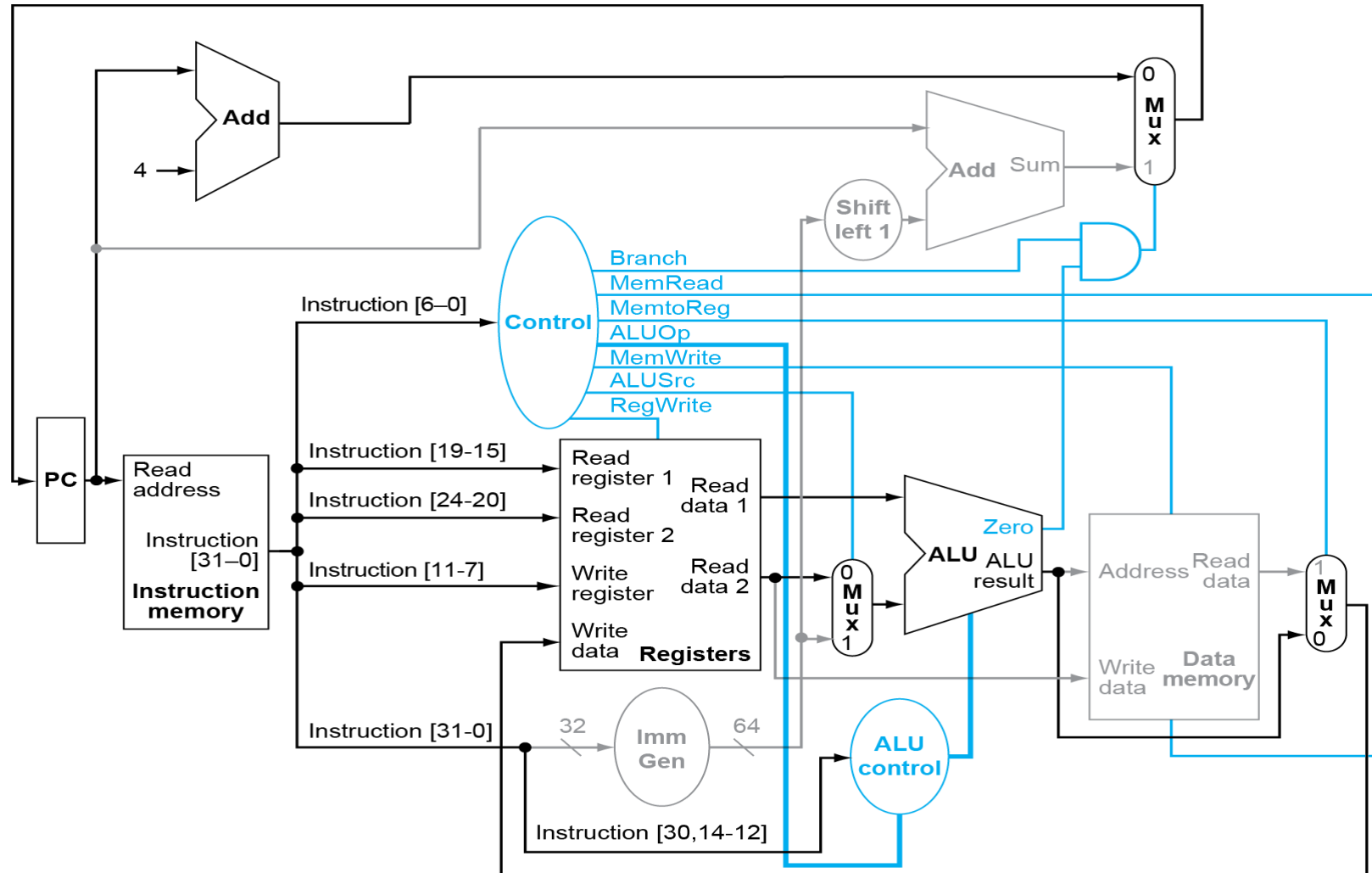
- Basic single-cycle CPU design, continued
- Pipelining idea and challenges

Recall our basic RISC-V CPU: datapath w/ control



R-Type Inst

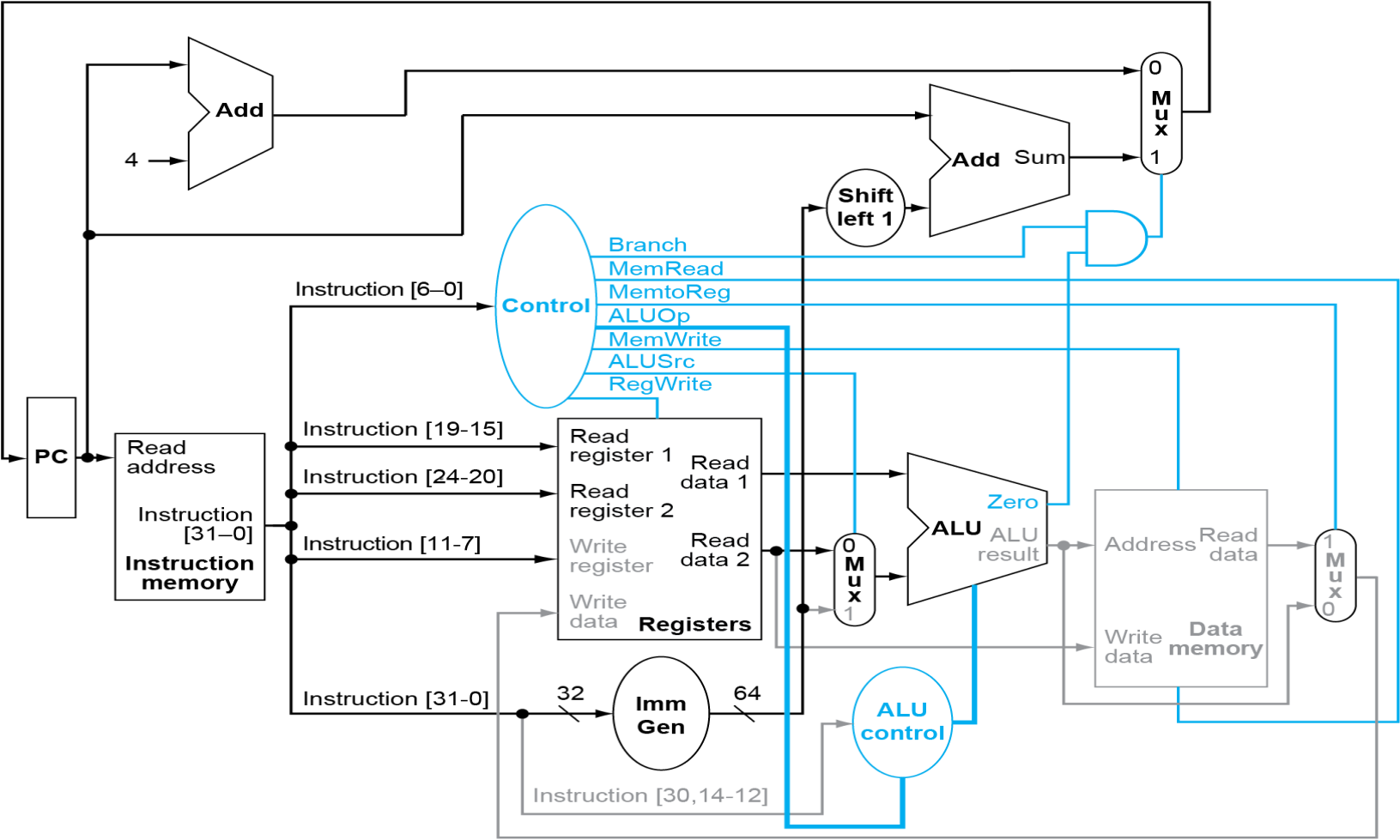
add x5, x6, x7



opcode	ALUOp	funct7	funct3	ALU control
ld	00	xxxxxxx	xxx	0010
sd	00	xxxxxxx	xxx	0010
beq	01	xxxxxxx	xxx	0110
R-type	10	000000	000	0010 (add)
		010000	000	0110 (sub)
		000000	111	0000 (AND)
		000000	110	0001 (OR)

BEQ Instruction

beq x5, x6, 100

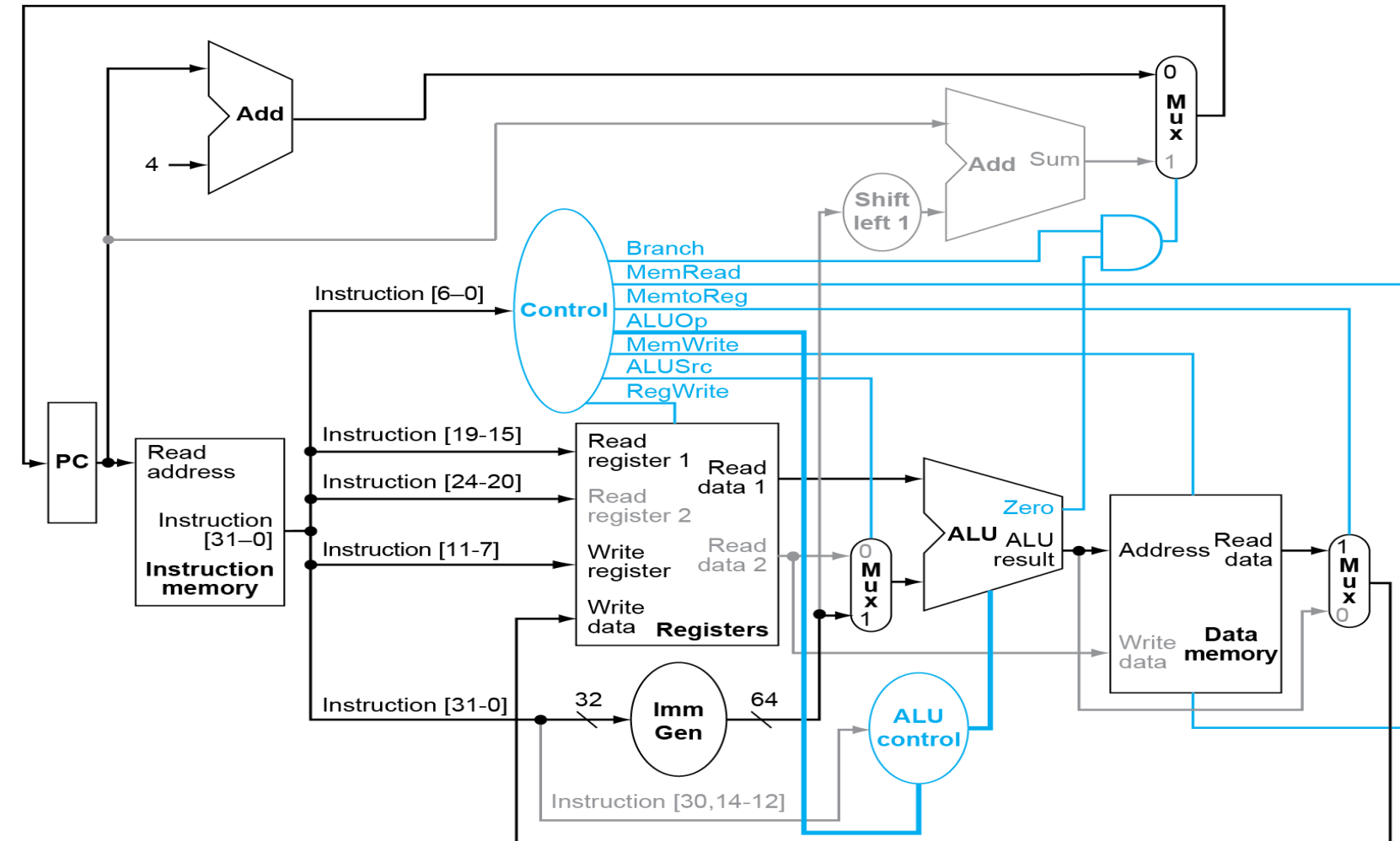


opcode	ALUOp	func7	func3	ALU control
ld	00	xxxxxxx	xxx	0010
sd	00	xxxxxxx	xxx	0010
beq	01	xxxxxxx	xxx	0110
R-type	10	000000	000	0010 (add)
		010000	000	0110 (sub)
		000000	111	0000 (AND)
		000000	110	0001 (OR)

ld x5, 40(x6)

Load Inst

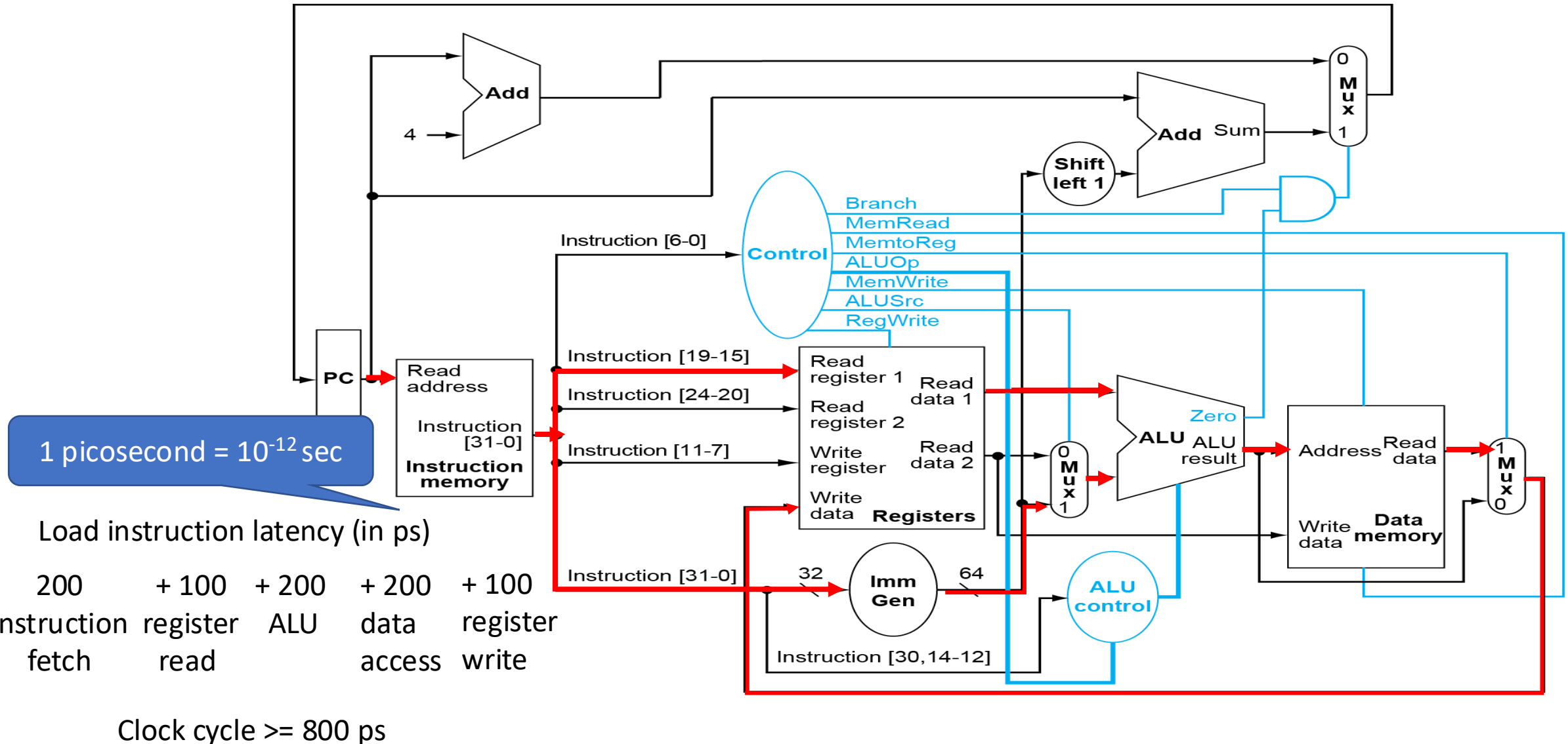
immediate	rs1	funct3	rd	opcode
-----------	-----	--------	----	--------



opcode	ALUOp	func7	func3	ALU control
ld	00	xxxxxxx	xxx	0010
sd	00	xxxxxxx	xxx	0010
beq	01	xxxxxxx	xxx	0110
R-type	10	000000	000	0010 (add)
		010000	000	0110 (sub)
		000000	111	0000 (AND)
		000000	110	0001 (OR)

Basic CPU must finish an instruction in one clock cycle

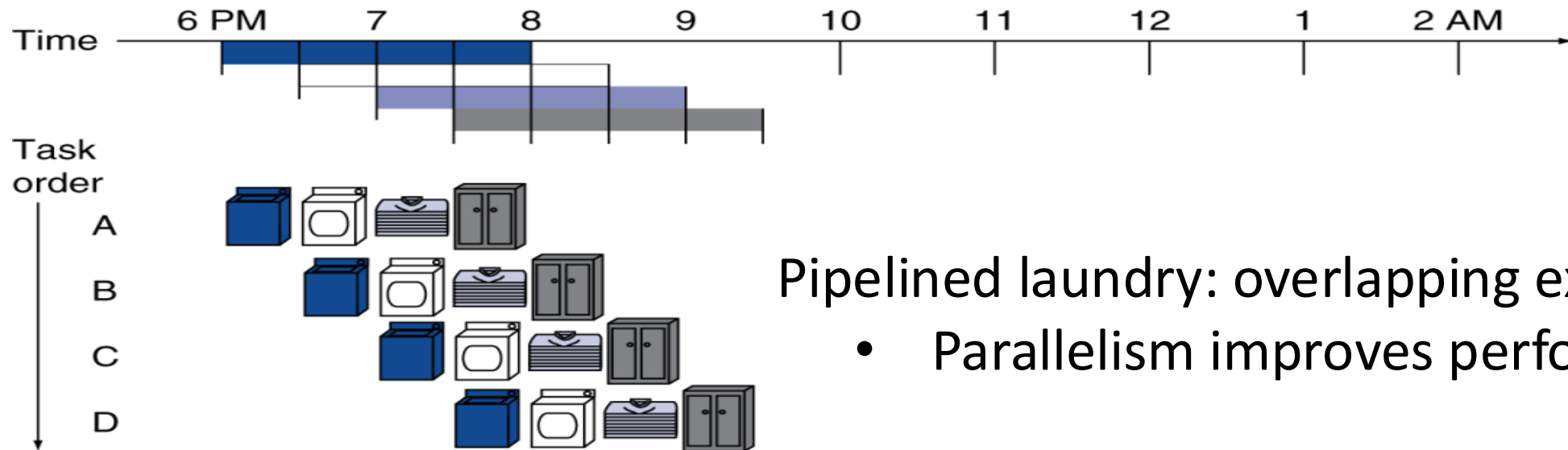
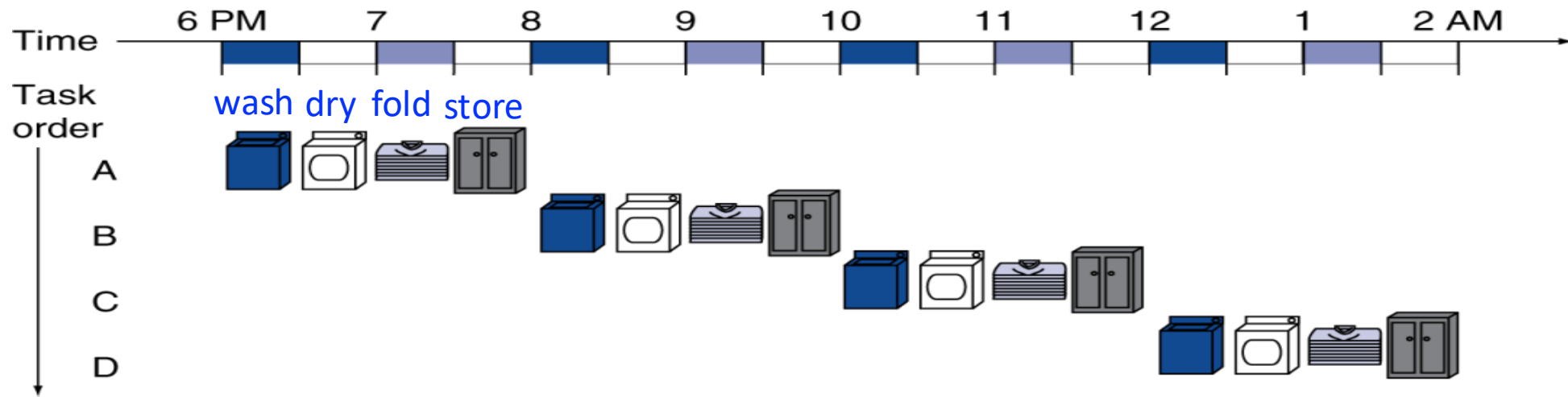
→ use a “slow” clock



Our basic design is slow

- Longest delay determines clock period
 - Critical path: load instruction
 - Instruction memory → register file → ALU → data memory → register file
- Not feasible to vary clock period for different instructions
- Next: improve performance by pipelining

Pipelining: a laundry analogy



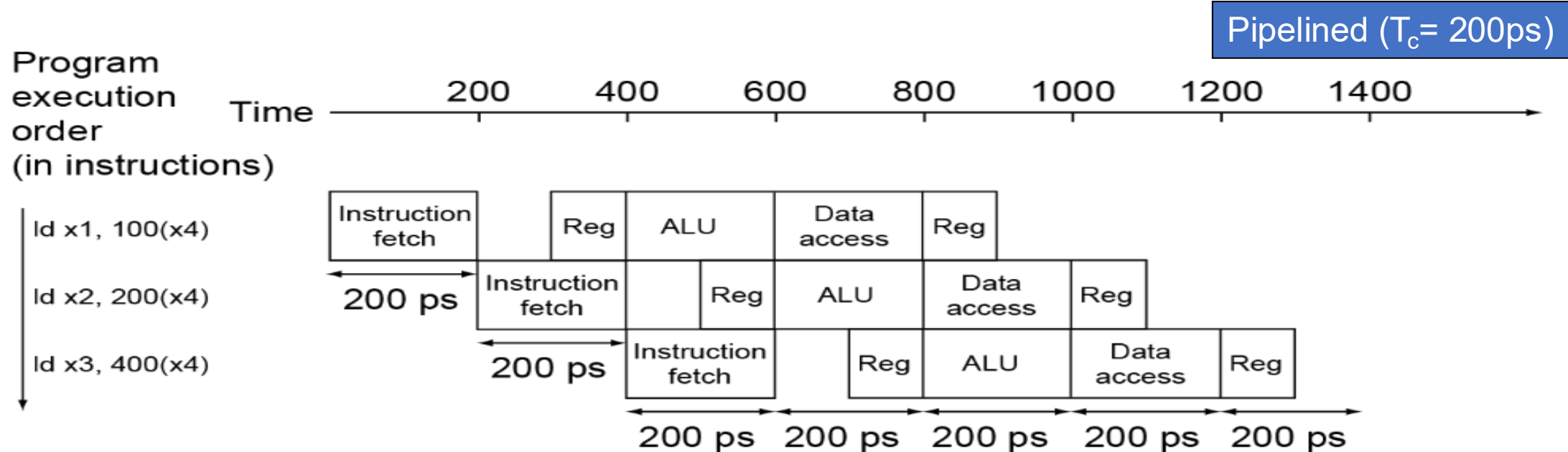
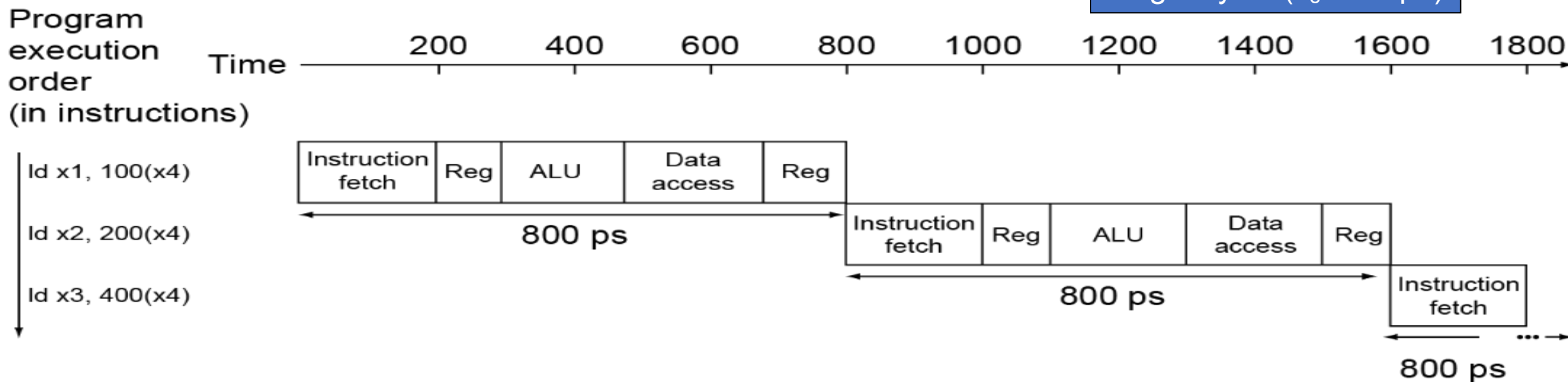
Pipelined laundry: overlapping execution

- Parallelism improves performance

RISC-V Pipeline

- Five stages:
 1. IF: Instruction fetch from memory
 2. ID: Instruction decode & register read
 3. EX: Execute operation or calculate address
 4. MEM: Access memory operand
 5. WB: Write result back to register

Pipeline Performance



Pipeline Speedup

- Pipelining increases throughput (instructions/sec)
 - Latency (time for each instruction) does not decrease
- If all stages are balanced (i.e., all take the same time)
 - $\text{throughput}_{\text{pipelined}} = \text{number-of-stages} * \text{throughput}_{\text{nonpipelined}}$
 - If not balanced, speedup is less



Throughput = $1/(\text{time between instructions})$

Pipelining and ISA Design

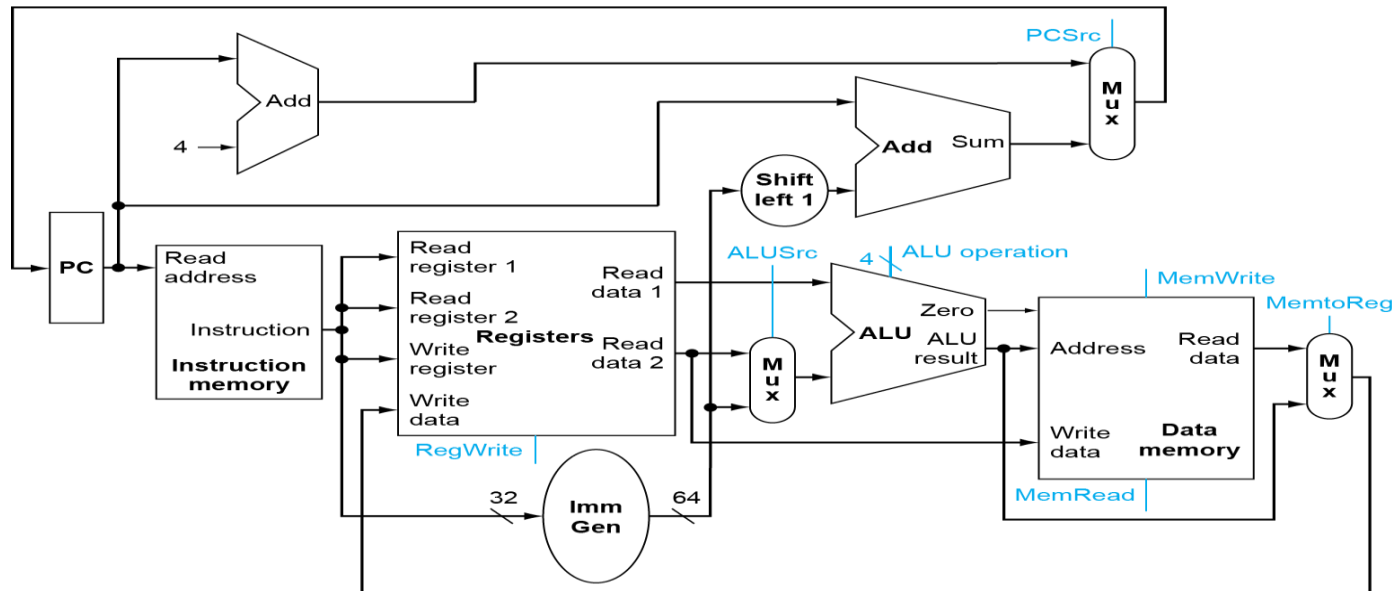
- RISC-V ISA is designed to simplify pipelined CPU design
 - All instructions are 32-bits
 - Easier to fetch and decode in one cycle
 - By contrast, x86: 1- to 17-byte instructions
 - Few and regular instruction formats
 - Can decode and read registers in one step
 - Load/store addressing
 - Can calculate address in 3rd stage, access memory in 4th stage

Pipeline challenges: hazards

- Situations that prevent starting next instruction in the next cycle
- Structure hazard
 - A required resource is busy
- Data hazard
 - Need to wait for previous instruction to complete its write
- Control hazard
 - Which instruction to execute depends on previous instruction

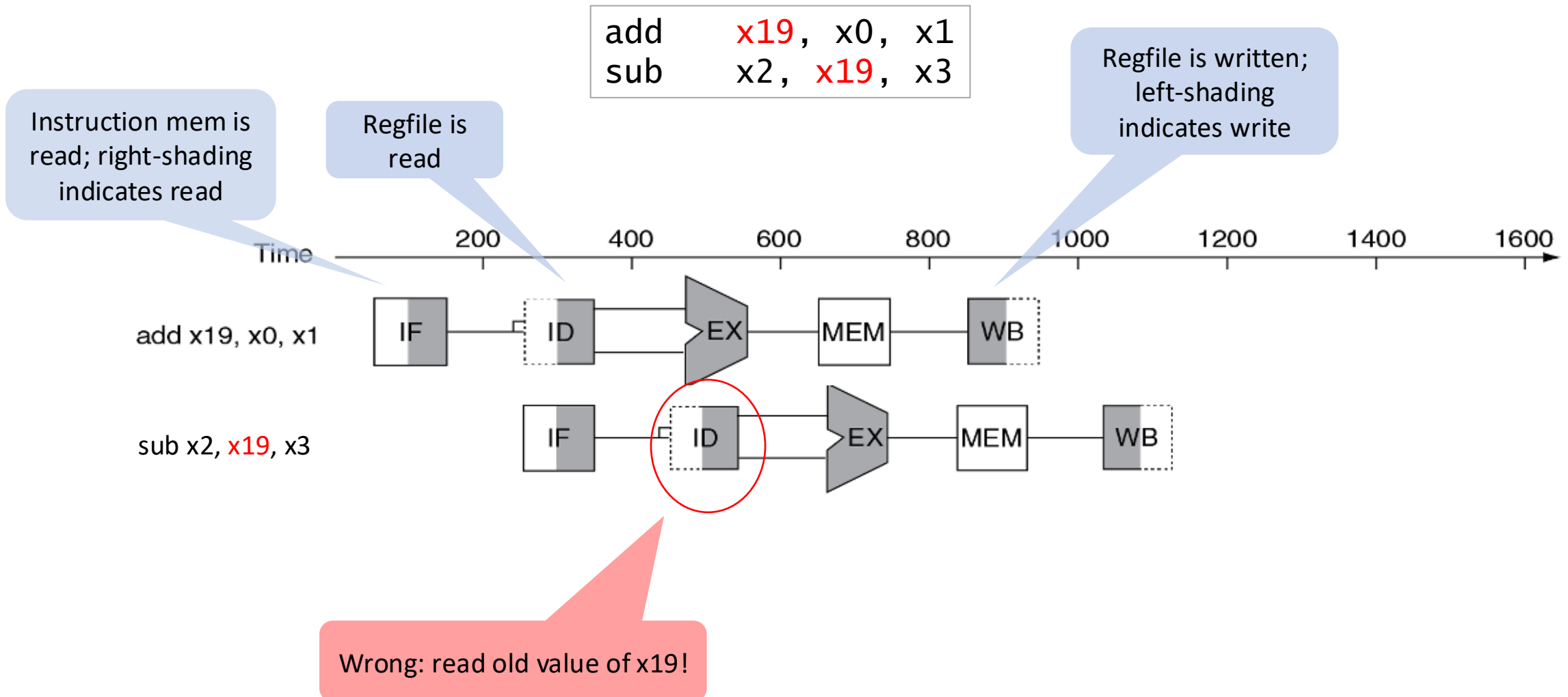
Structure Hazards

- Conflict use of a single resource
- Example: Suppose CPU only has a single on-chip memory module
 - Load/store requires data access
 - Instruction fetch would have to *stall* for that cycle
 - Would cause a pipeline “bubble”
- Solution: Use separate instruction/data memories



Data Hazards

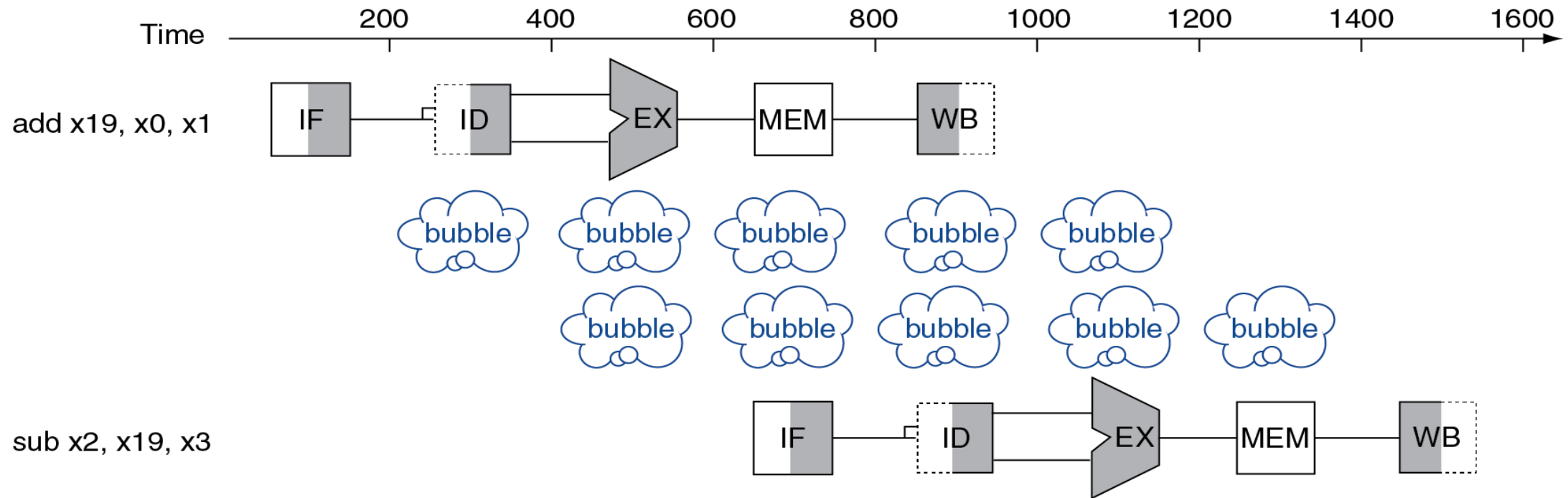
- An instruction depends on the previous instruction to complete its write



Data Hazards

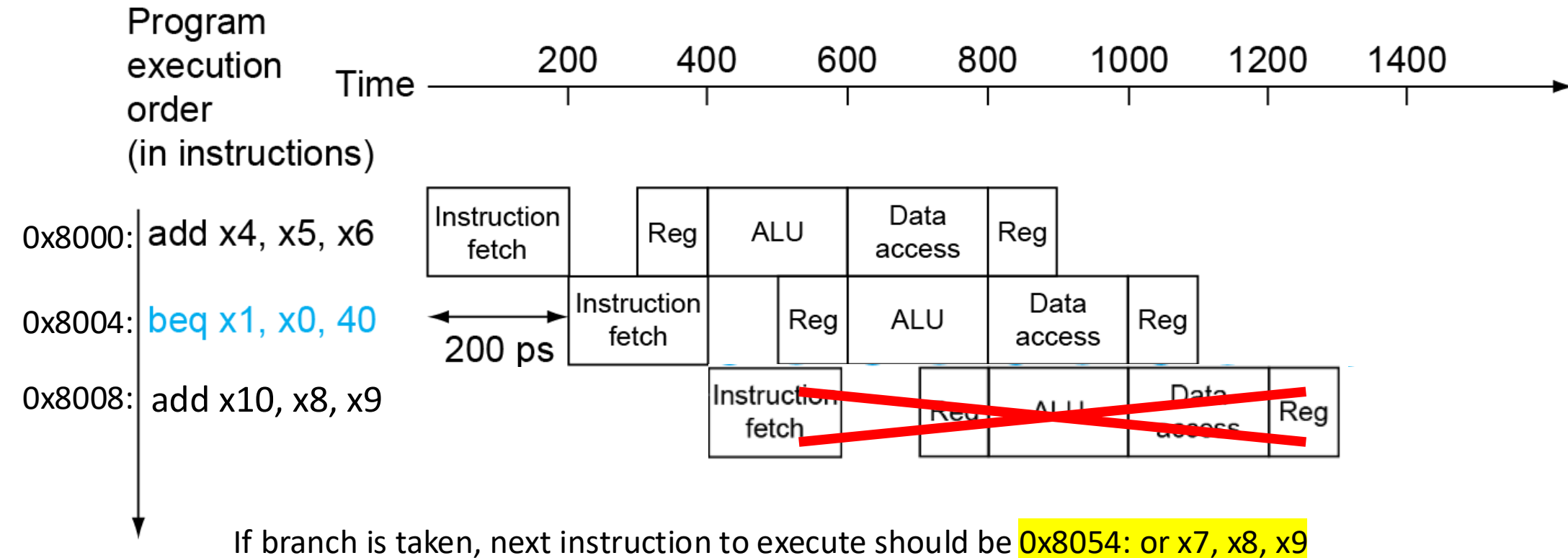
- An instruction depends on the previous instruction to complete its write

```
add    x19, x0, x1  
sub    x2, x19, x3
```



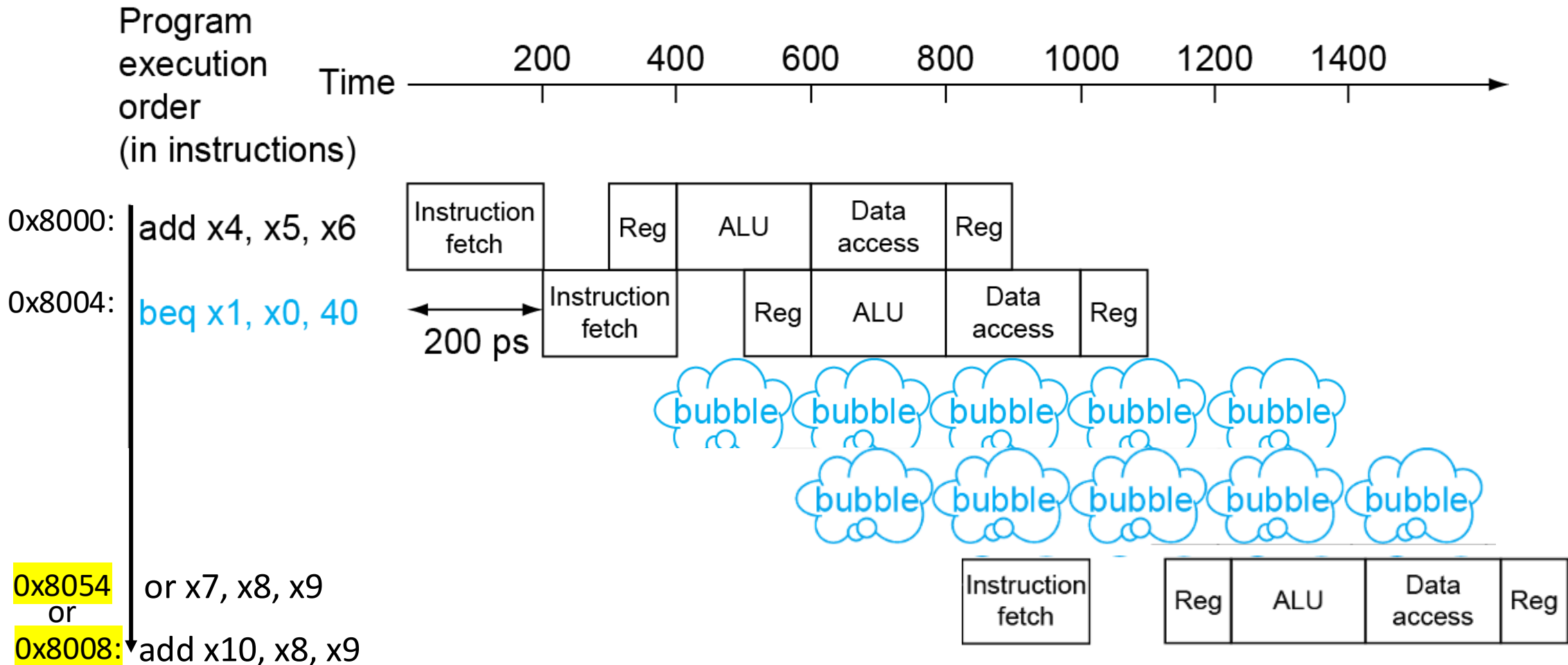
Control hazard

- Wait until branch outcome is determined before fetching next instruction



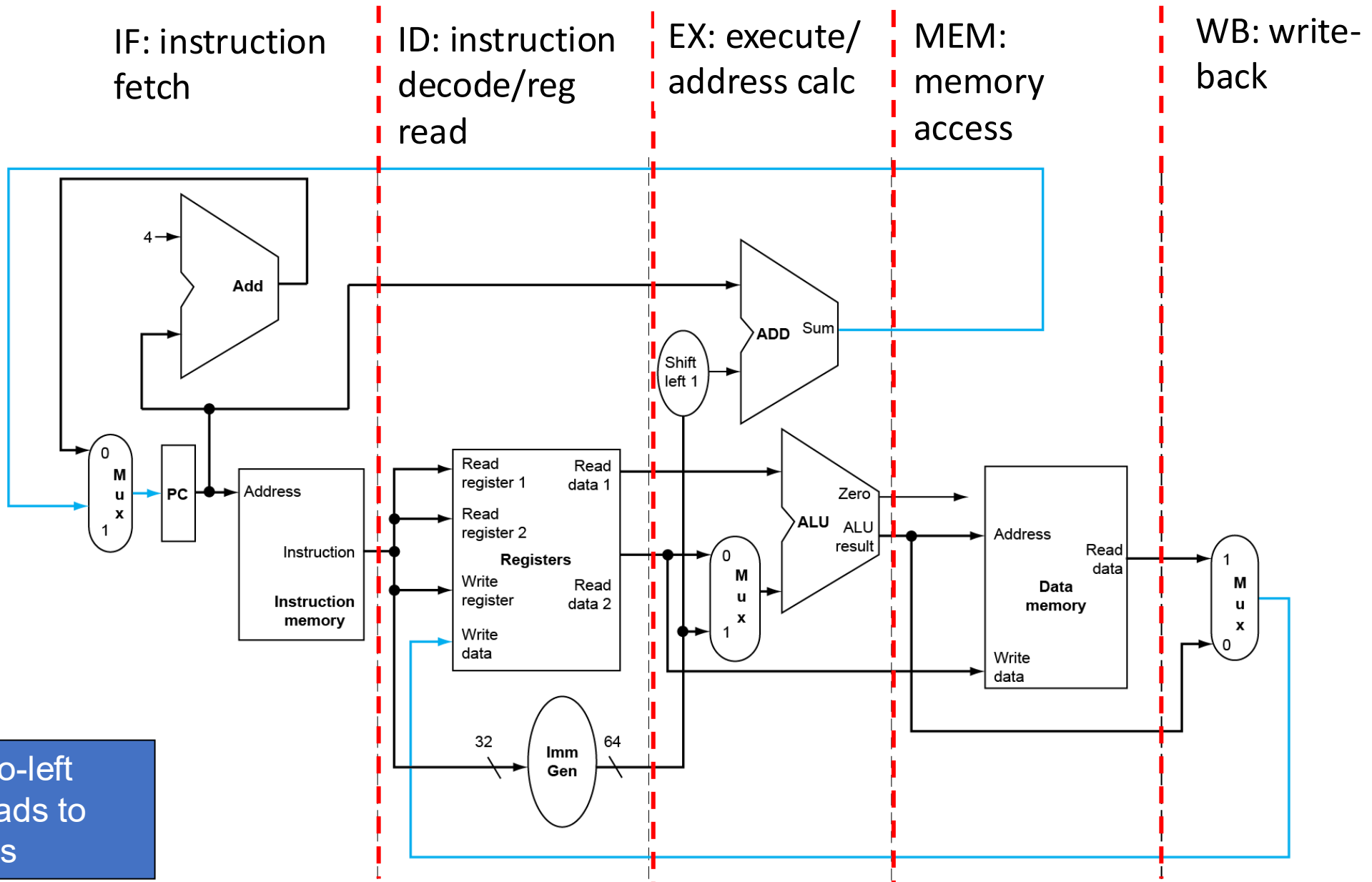
Control hazard

- Wait until branch outcome is determined before fetching next instruction



A basic pipelined RISC-V CPU

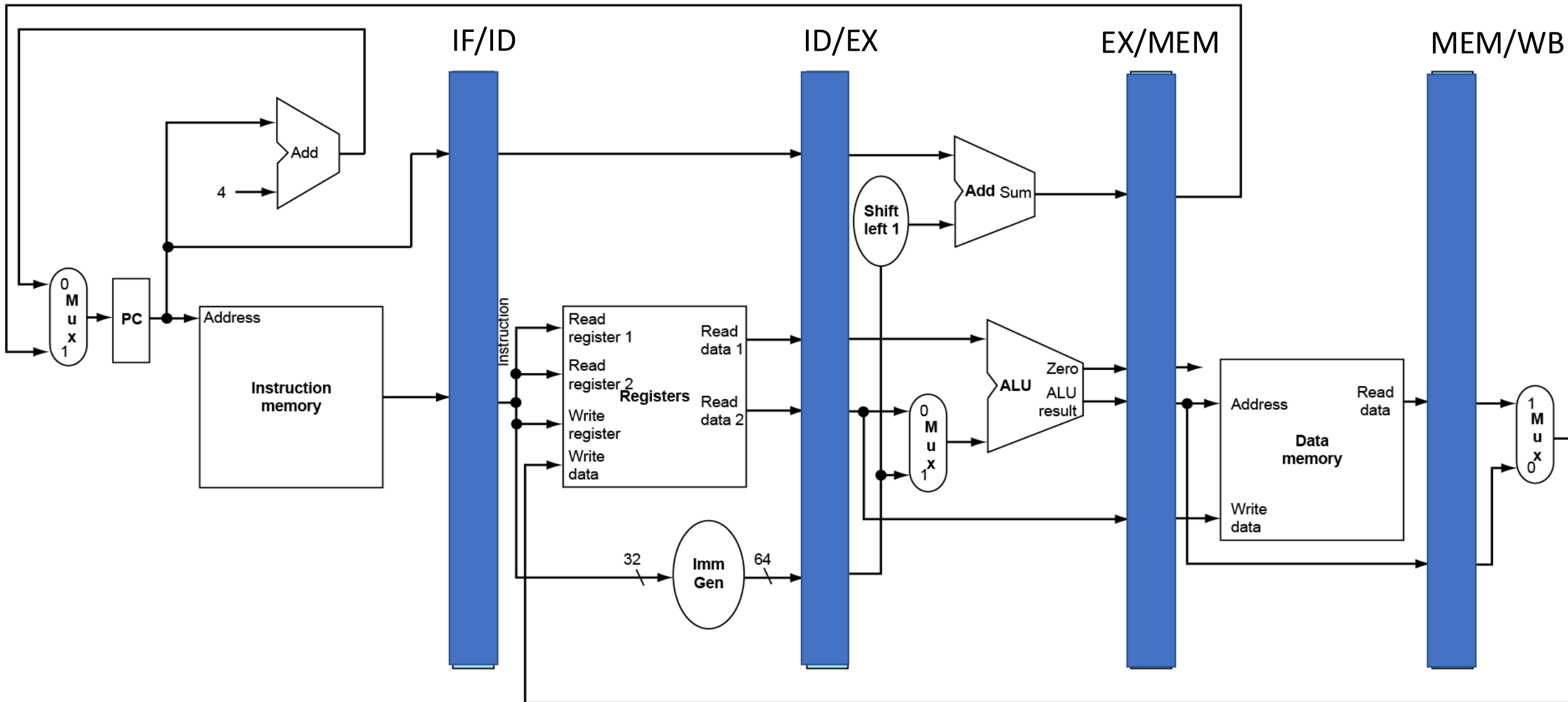
Pipelined Datapath



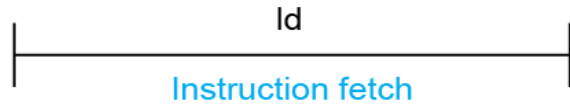
Right-to-left
flow leads to
hazards

Pipeline registers

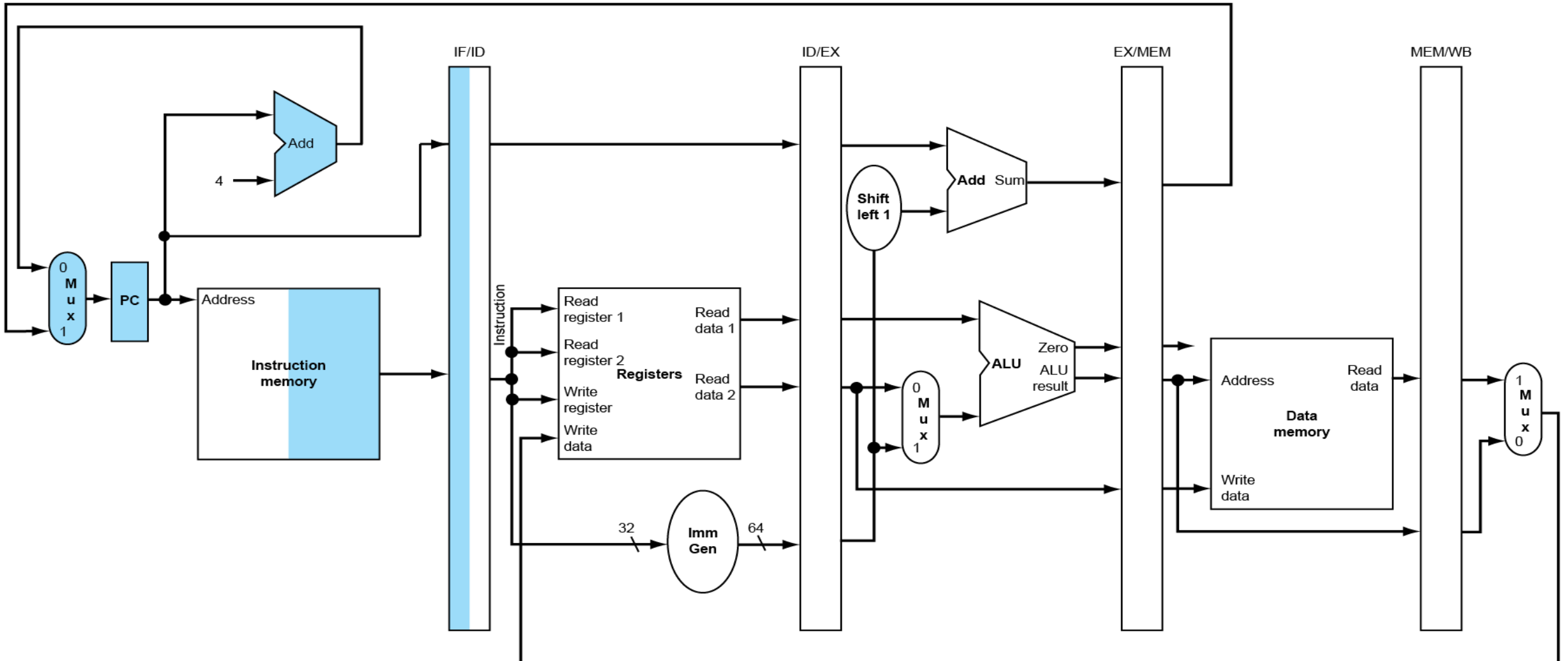
: needed to hold data produced in previous cycle



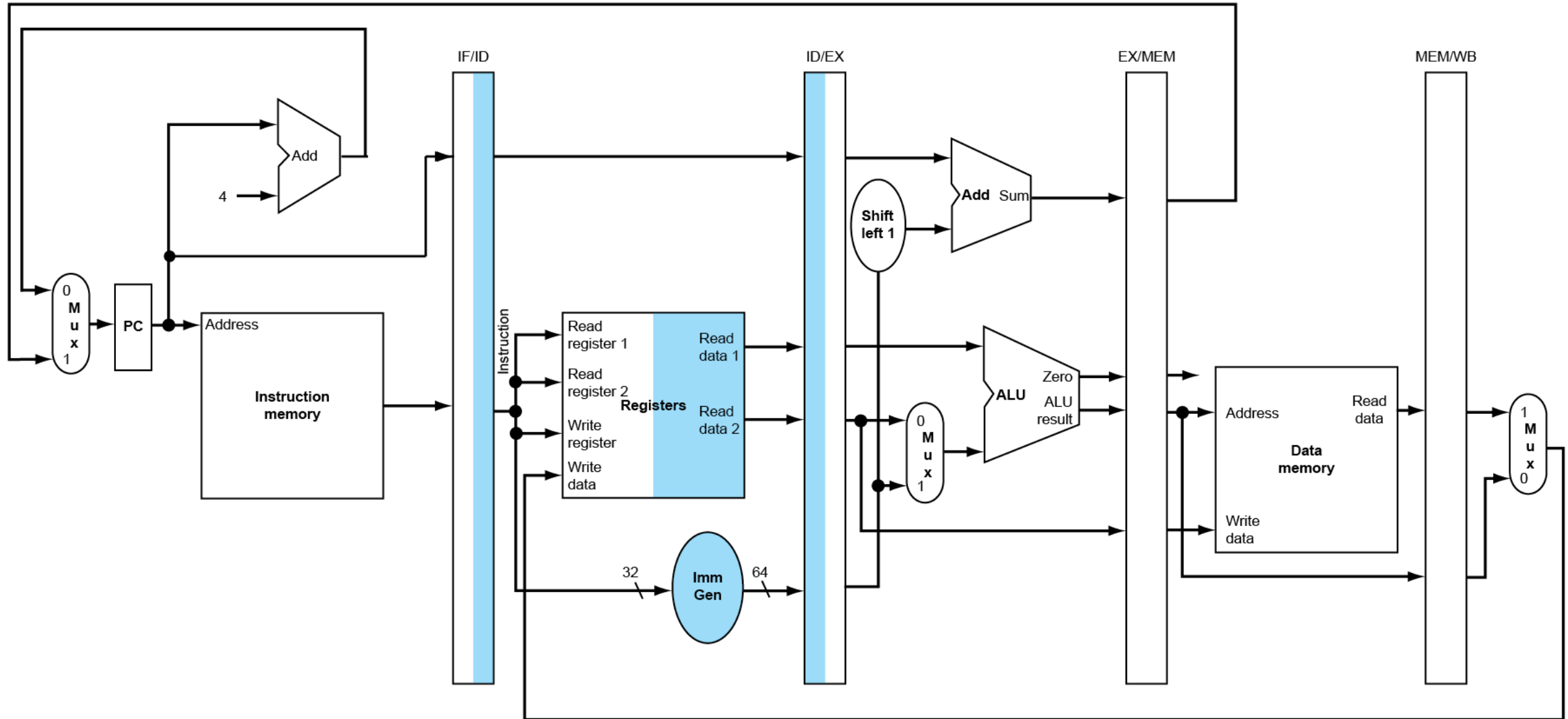
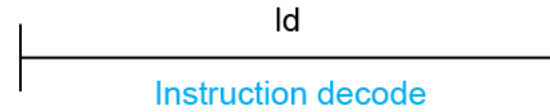
IF for Load, Store



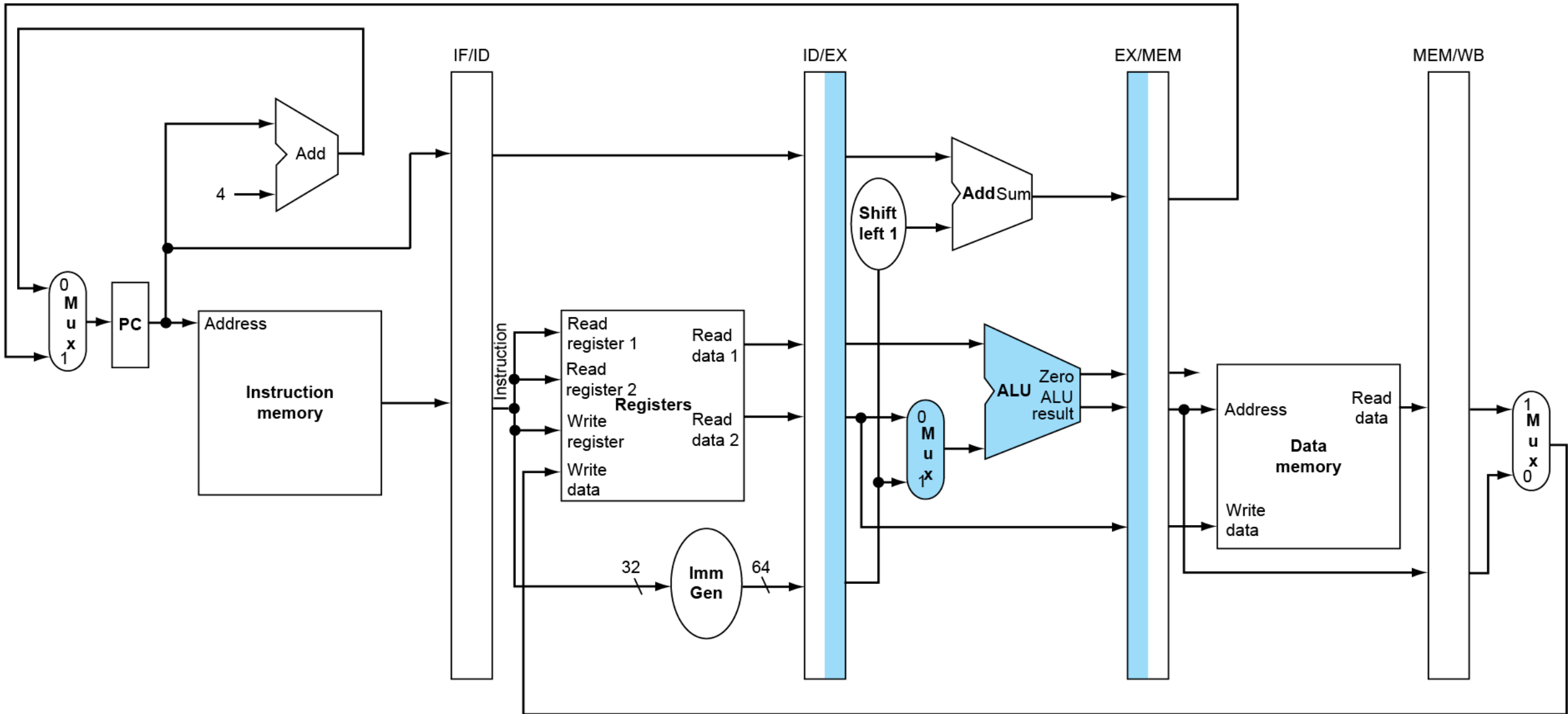
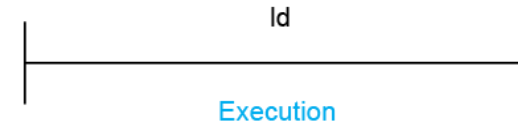
Single-clock-cycle diagram shows the state of an entire datapath during a single clock cycle



ID for Load, Store, ...

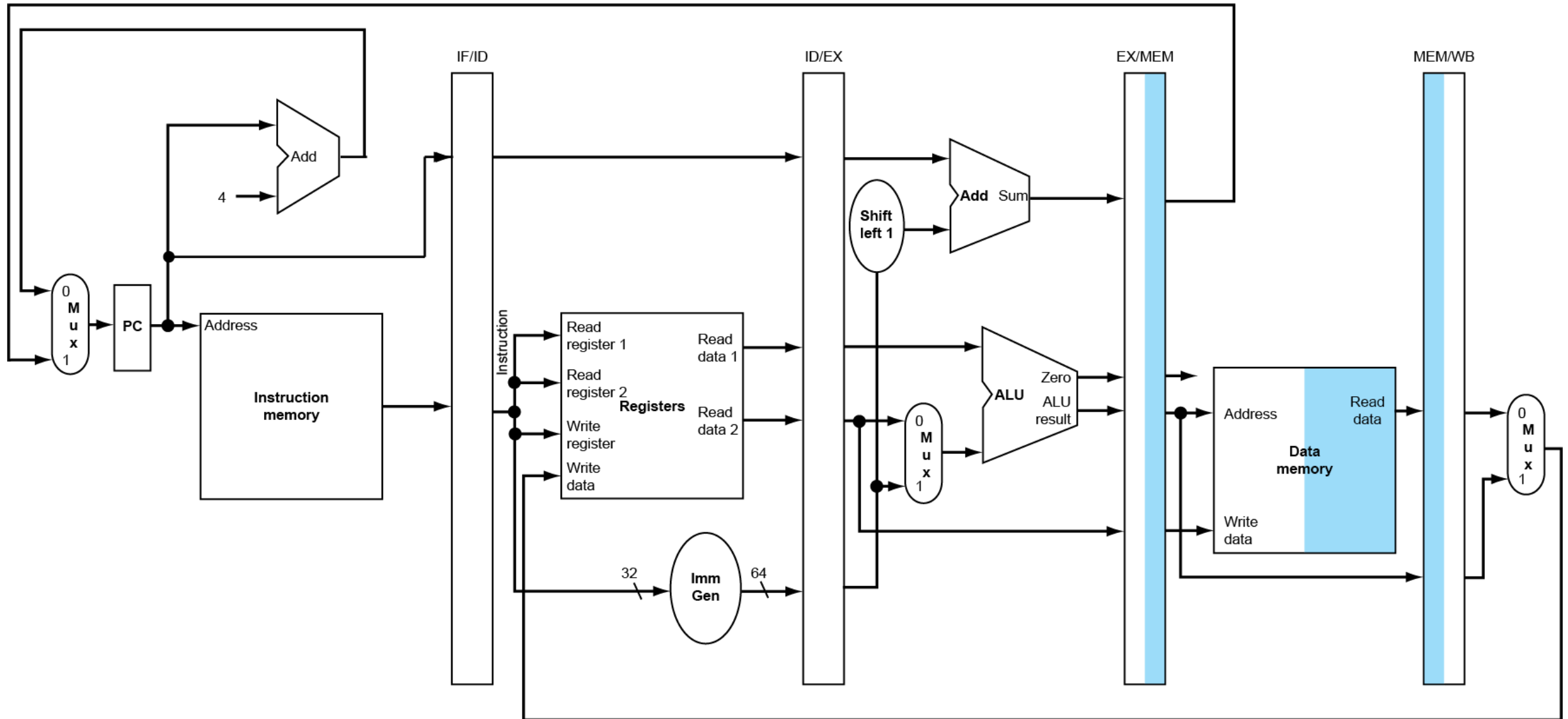


EX for Load



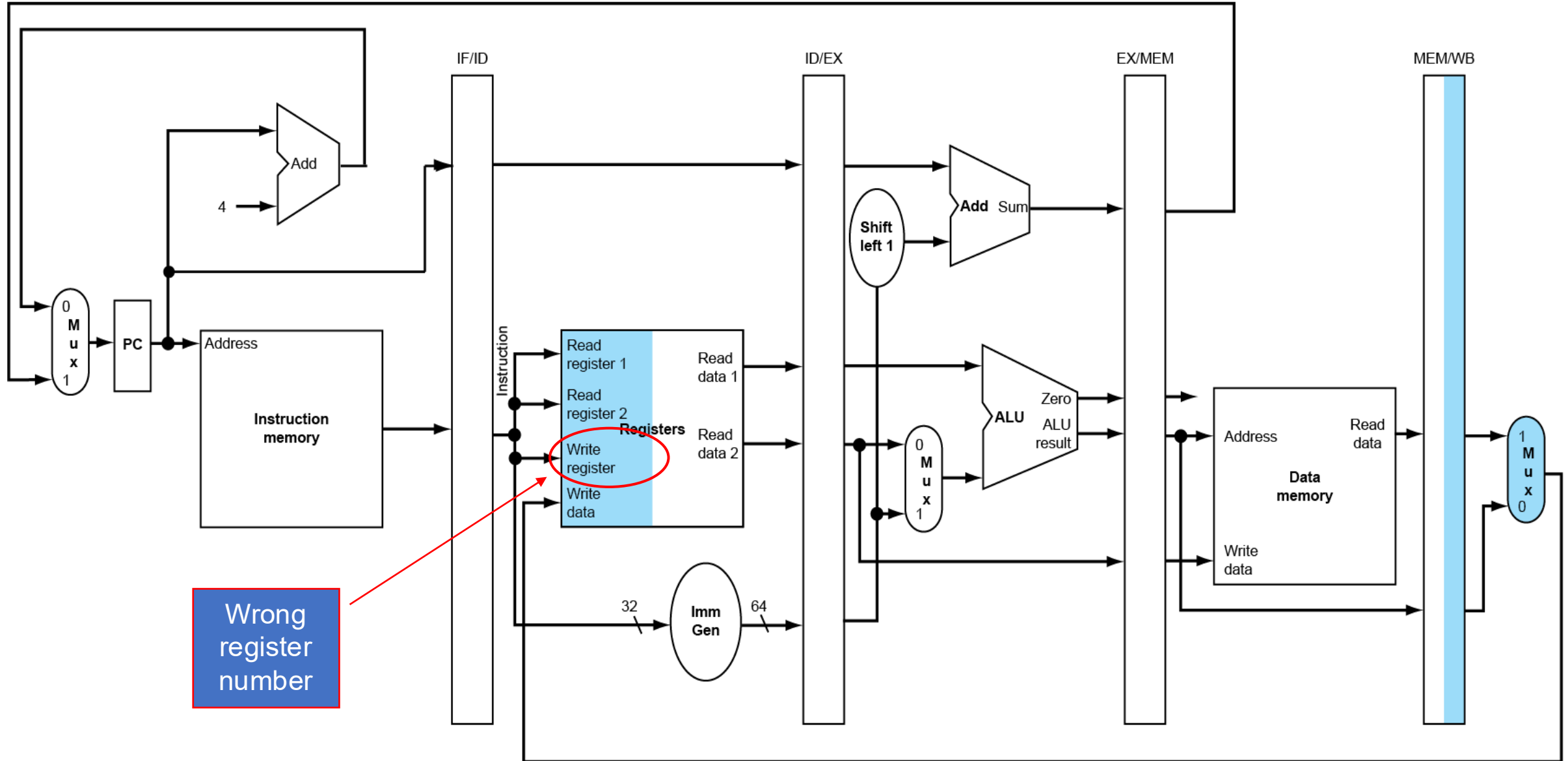
MEM for Load

Id
Memory

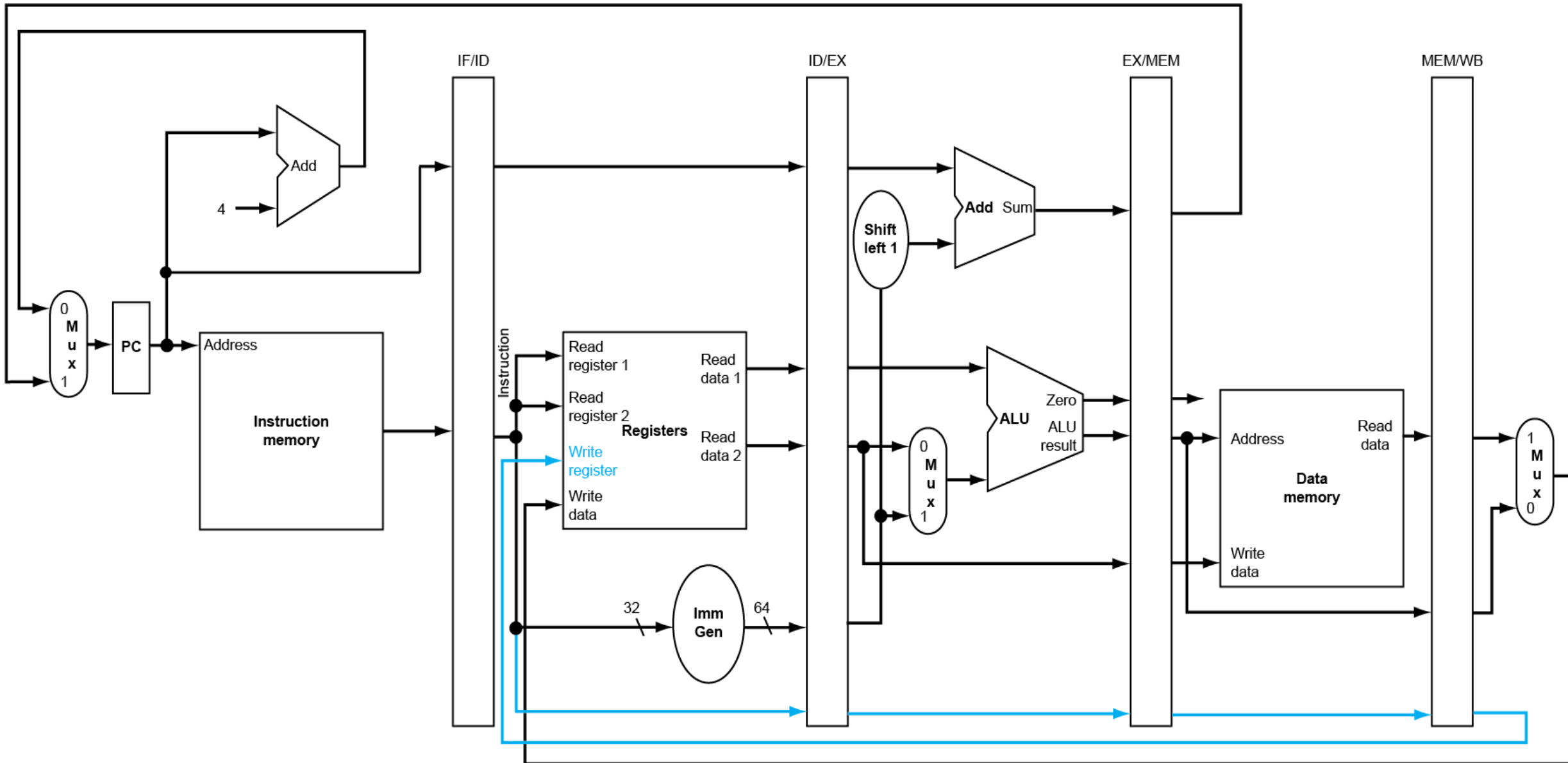


WB for Load

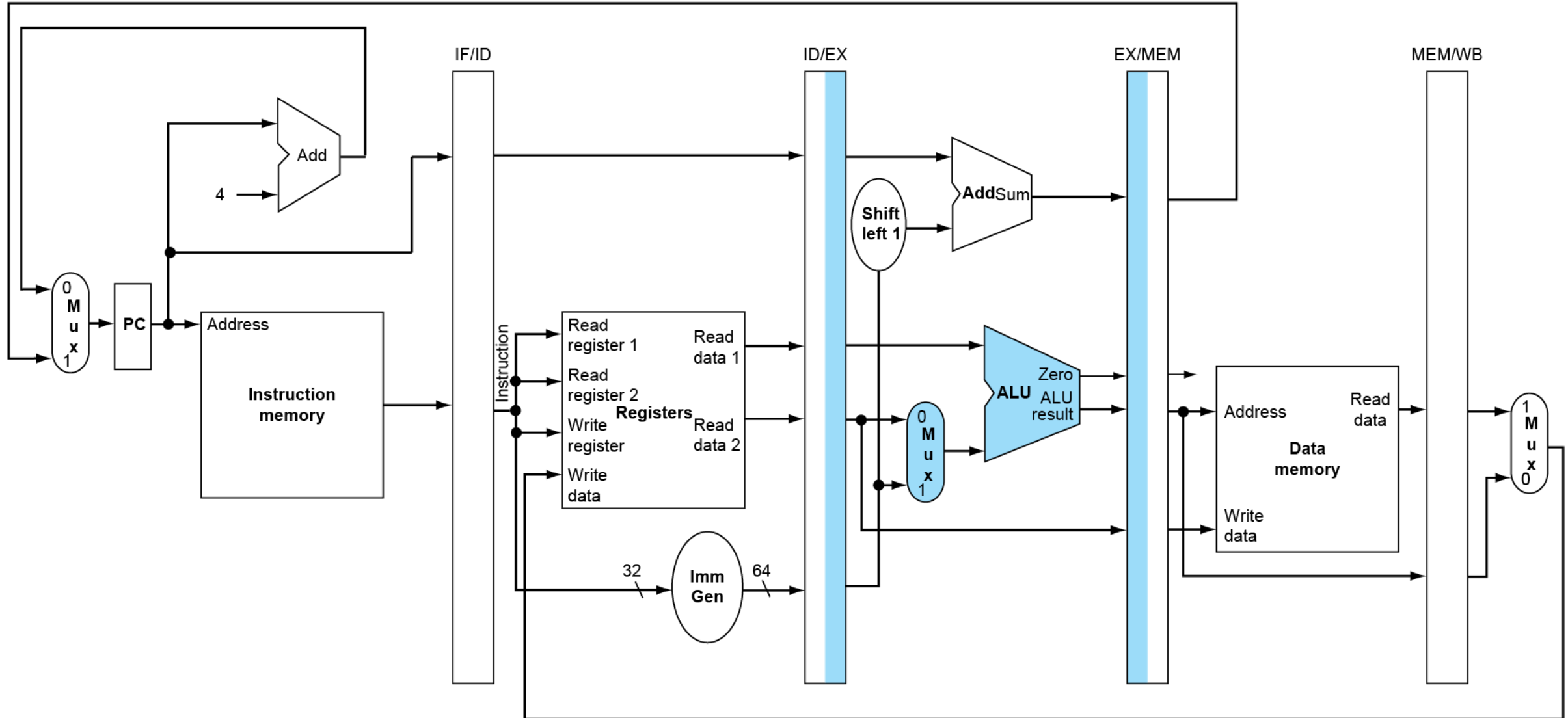
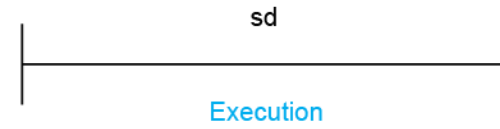
ld
Write-back



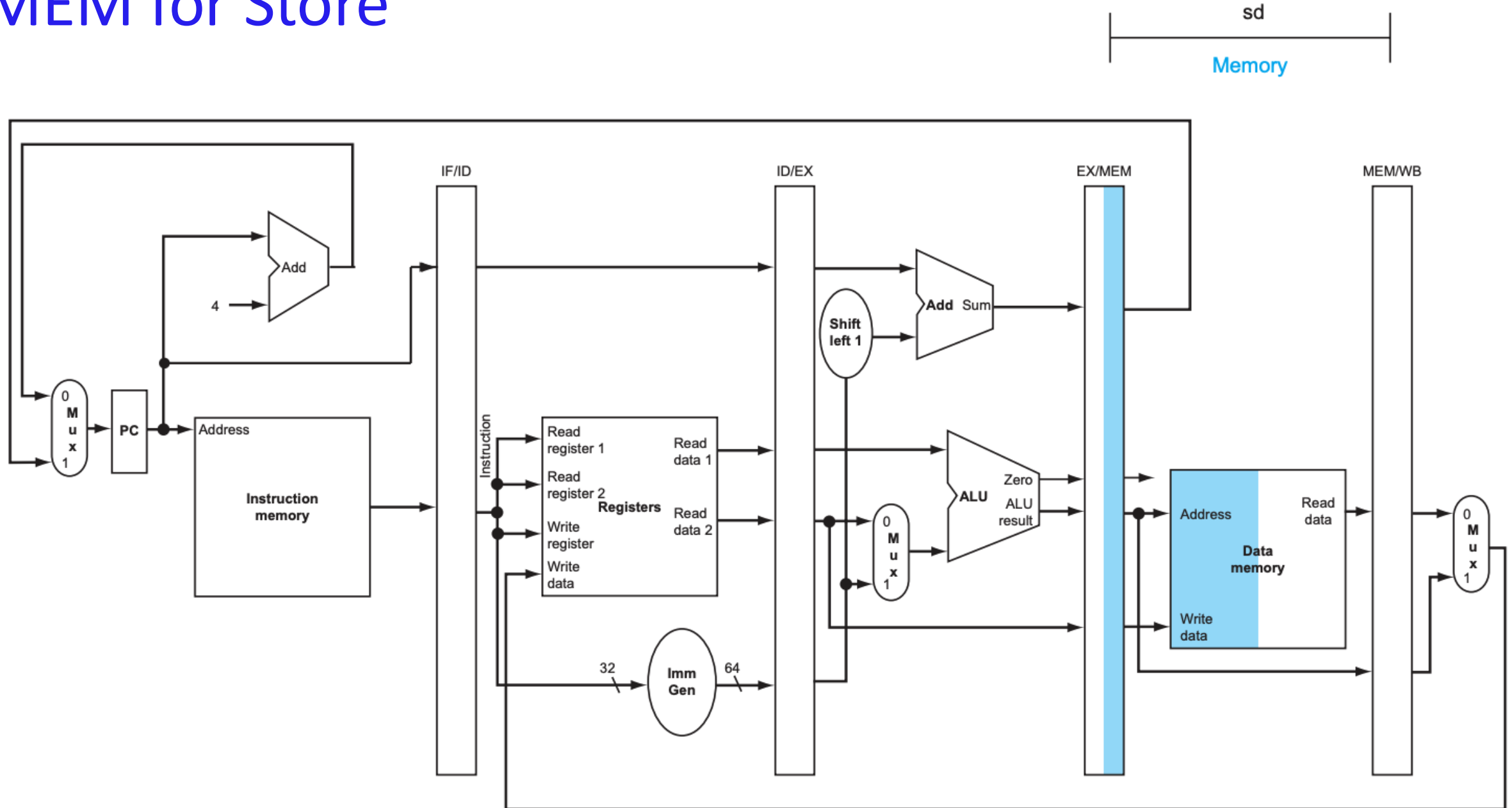
Corrected Datapath for Load



EX for Store

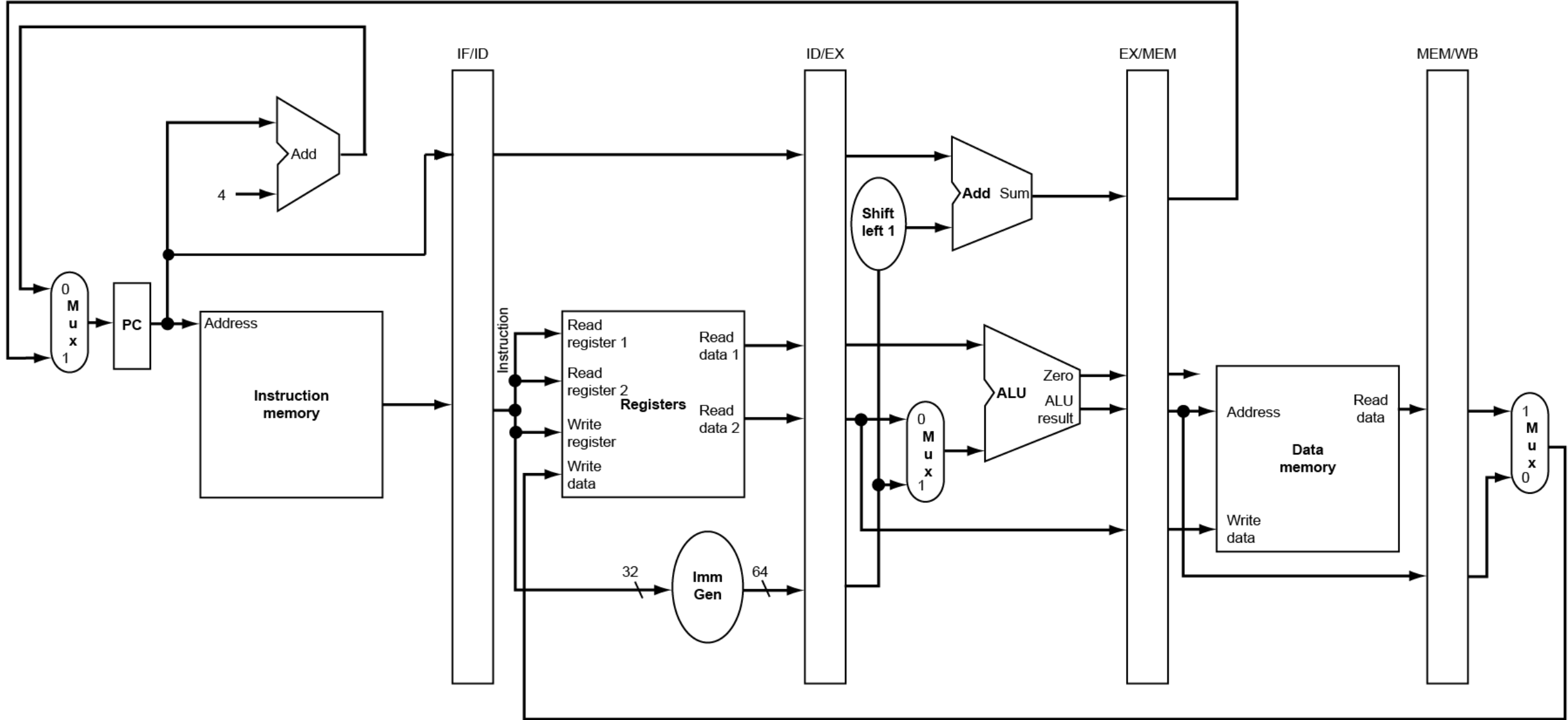


MEM for Store



WB for Store

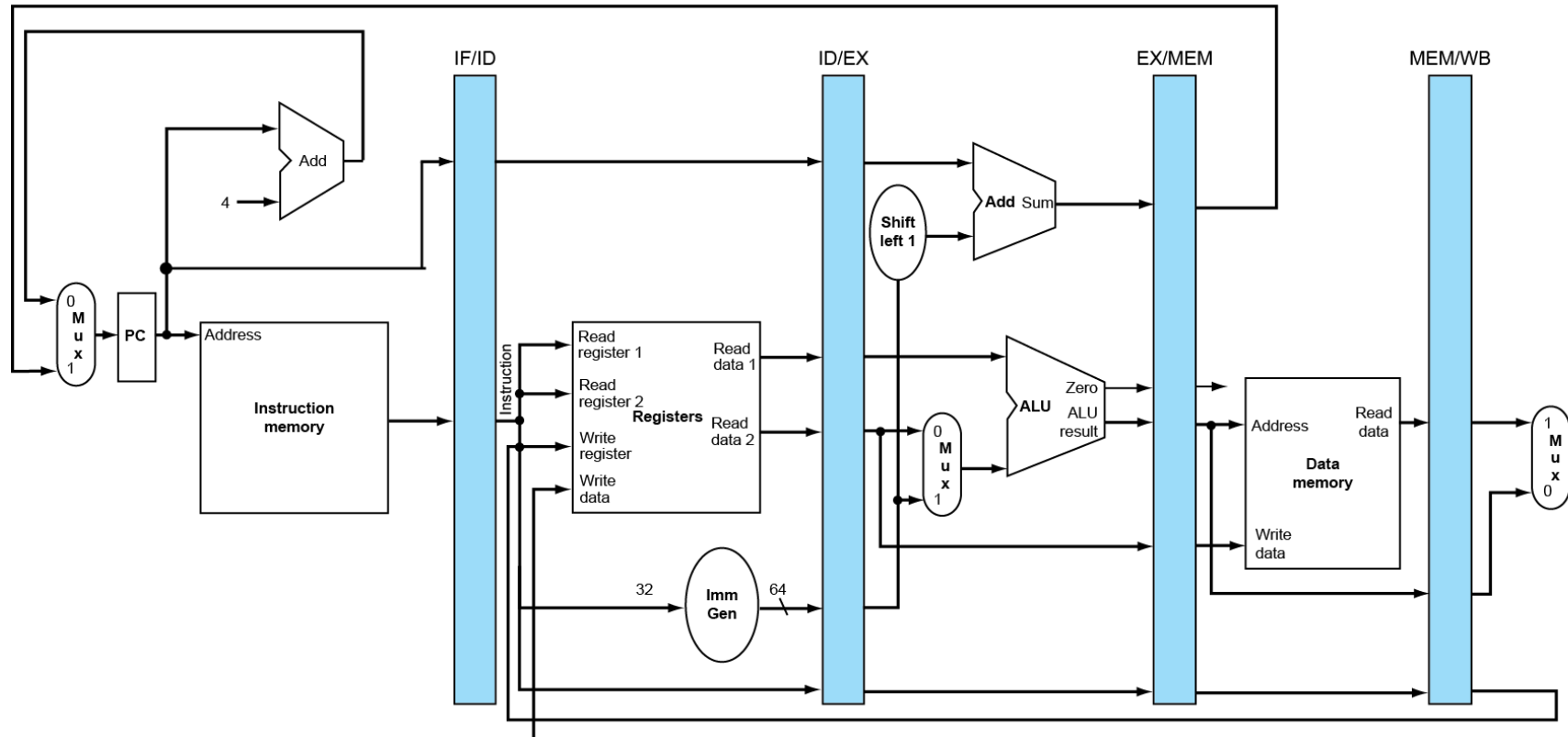
sd
Write-back



Single-Cycle Pipeline Diagram

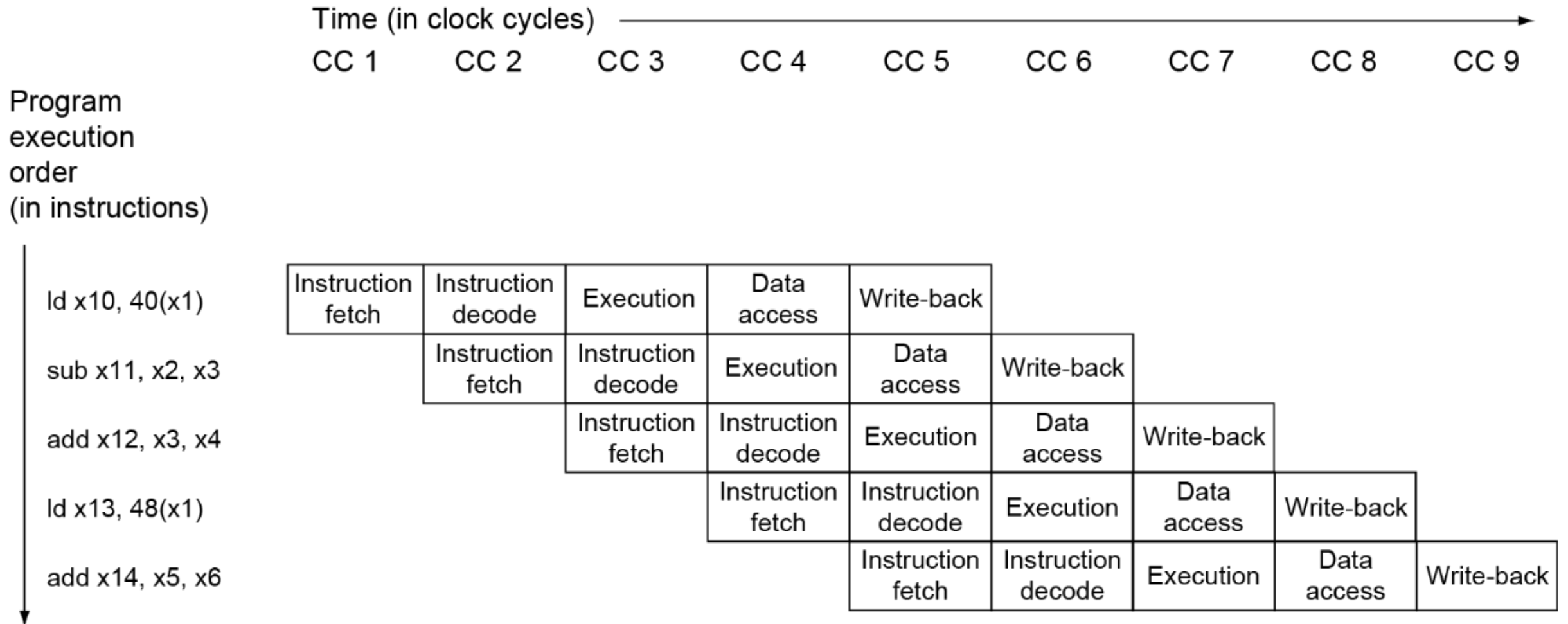
- State of pipeline in a given cycle

add x14, x5, x6	ld x13, 48(x1)	add x12, x3, x4	sub x11, x2, x3	ld x10, 40(x1)
Instruction fetch	Instruction decode	Execution	Memory	Write-back



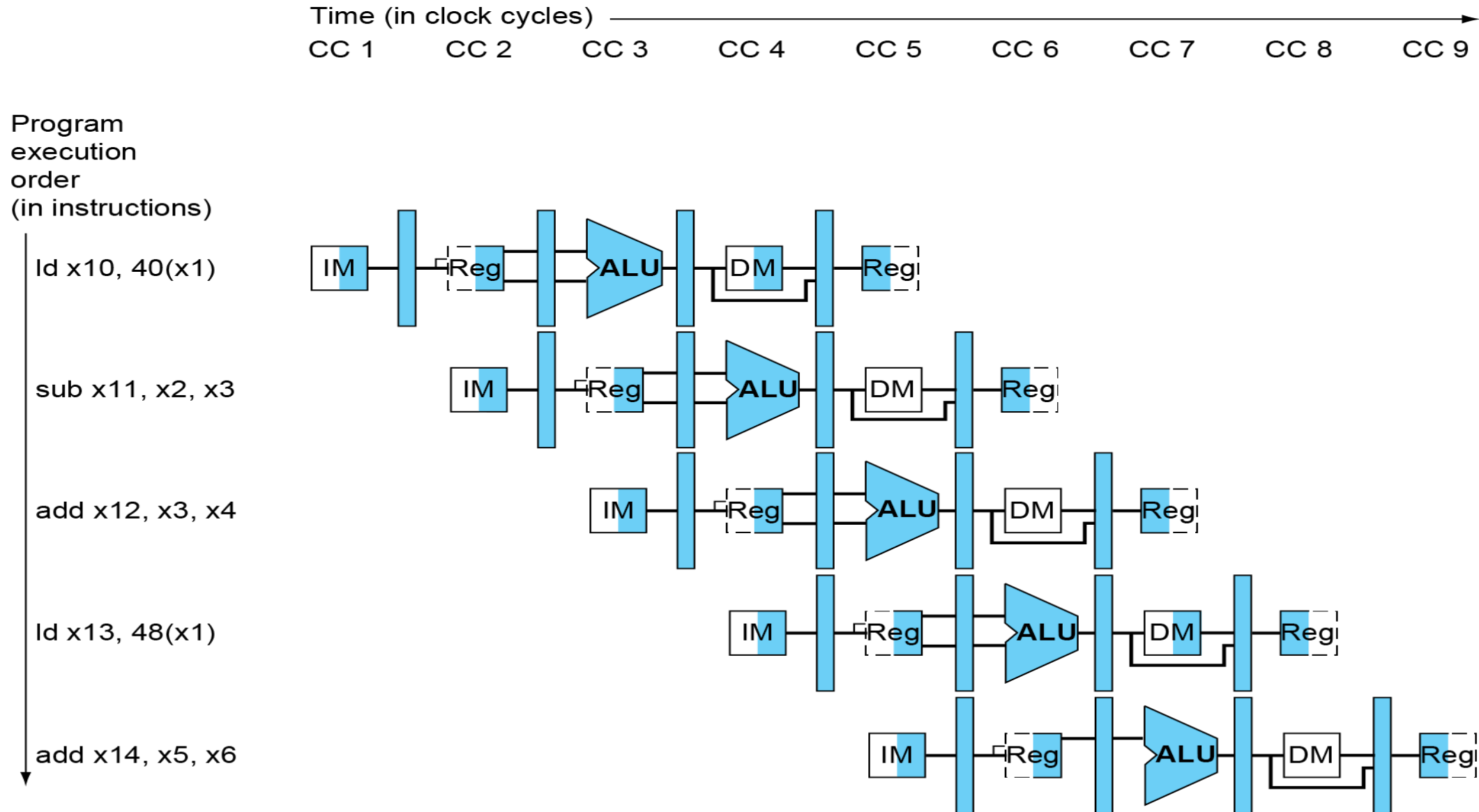
Multi-Cycle Pipeline Diagram

- Traditional form



Multi-Cycle Pipeline Diagram

- Form showing resource usage



Summary

- Basic single-cycle CPU design
 - Data path vs. control path
 - Clock frequency is limited by the longest delay
- Basic 5-stage pipelined design:
 - Main idea: Parallel processing of different stages of an instruction's execution
 - RISC-V 5-stage pipeline (IF, ID, EXE, MEM, WB)
 - Pipeline hazards: structure, data, control